

Bài toán liệt kê

Câu 1. Liệt kê các xâu nhị phân độ dài n bằng hai phương pháp: sinh tuần tự và quay lui

Ta cần liệt kê tất cả các xâu nhị phân độ dài n , tức là các xâu

$$x_1x_2\cdots x_n \quad \text{với } x_i \in \{0, 1\}.$$

Tổng số xâu nhị phân độ dài n là

$$2^n.$$

1. Phương pháp sinh tuần tự

Ý tưởng của phương pháp sinh tuần tự là xem mỗi xâu nhị phân độ dài n như một cấu hình gồm n vị trí, mỗi vị trí nhận giá trị 0 hoặc 1. Bắt đầu từ cấu hình nhỏ nhất là

$$00\cdots 0,$$

sau đó sinh cấu hình kế tiếp theo thứ tự từ điển tăng dần cho đến cấu hình cuối cùng là

$$11\cdots 1.$$

Quy tắc sinh cấu hình kế tiếp

Giả sử đang có xâu nhị phân

$$x_1x_2\cdots x_n.$$

Để sinh xâu kế tiếp, ta thực hiện như sau:

- Tìm từ phải sang trái vị trí đầu tiên i sao cho $x_i = 0$.
- Đổi x_i thành 1.
- Đặt tất cả các phần tử phía sau nó, tức là $x_{i+1}, x_{i+2}, \dots, x_n$, thành 0.

Nếu không còn vị trí nào có giá trị 0, tức là xâu hiện tại là

$$11\cdots 1,$$

thì ta dừng vì đó là xâu cuối cùng.

Ví dụ với $n = 3$

Bắt đầu từ 000, các xâu được sinh lần lượt là:

$$000, 001, 010, 011, 100, 101, 110, 111.$$

Thuật toán

Khởi tạo $x_1 = x_2 = \dots = x_n = 0$

Lặp:

 In ra $x_1x_2\dots x_n$

 Tìm i từ n về 1 sao cho $x_i = 0$

 Nếu không tìm thấy thì dừng

 Đặt $x_i = 1$

 Với mọi $j = i+1, \dots, n$ thì đặt $x_j = 0$

Nhận xét

Phương pháp này sinh trực tiếp từng xâu theo đúng thứ tự. Mỗi lần sinh được đúng một cấu hình mới, do đó liệt kê được đầy đủ 2^n xâu nhị phân độ dài n .

2. Phương pháp quay lui

Ý tưởng của phương pháp quay lui là xây dựng xâu từ trái sang phải. Tại mỗi vị trí k ($1 \leq k \leq n$), ta thử lần lượt hai khả năng:

$$x_k = 0 \quad \text{hoặc} \quad x_k = 1.$$

Sau khi gán giá trị cho vị trí hiện tại, ta tiếp tục xét vị trí kế tiếp. Khi đã gán đủ n vị trí thì thu được một xâu nhị phân hoàn chỉnh và in ra.

Mô hình đệ quy

Gọi Try(k) là thủ tục gán giá trị cho vị trí thứ k .

- Nếu $k > n$ thì ta đã tạo xong một xâu nhị phân độ dài n , in ra xâu đó.

- Ngược lại, thử:

$$x_k = 0,$$

 rồi gọi Try($k+1$); sau đó thử

$$x_k = 1,$$

 rồi gọi Try($k+1$).

Thuật toán

Try(k):

 For v in $\{0,1\}$:

$x_k = v$

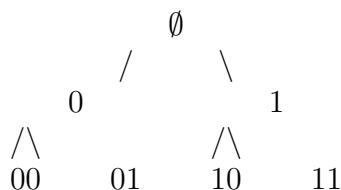
 Nếu $k = n$ thì in ra $x_1x_2\dots x_n$

 Ngược lại gọi Try($k+1$)

Ban đầu gọi Try(1).

Ví dụ với $n = 3$

Cây quay lui có dạng:



Đi tiếp đến độ sâu 3, ta thu được các xâu:

000, 001, 010, 011, 100, 101, 110, 111.

Nhận xét

Phương pháp quay lui phù hợp với các bài toán liệt kê tổ hợp, chỉnh hợp, hoán vị, xâu nhị phân, ... Ưu điểm lớn là dễ mở rộng khi bài toán có thêm điều kiện ràng buộc. Chẳng hạn, nếu chỉ cần liệt kê các xâu nhị phân độ dài n không có hai số 1 liên tiếp, ta chỉ cần kiểm tra điều kiện ngay trong quá trình quay lui.

3. So sánh hai phương pháp

Có thể liệt kê các xâu nhị phân độ dài n bằng:

- phương pháp sinh tuần tự: bắt đầu từ $00 \cdots 0$ và sinh dần đến $11 \cdots 1$;
- phương pháp quay lui: tại mỗi vị trí chọn một trong hai giá trị 0 hoặc 1 cho đến khi đủ n vị trí.

Cả hai đều cho ra toàn bộ 2^n xâu nhị phân độ dài n .

Câu 2. Liệt kê các tập con k phần tử của $\{1, 2, \dots, n\}$

Ta cần liệt kê tất cả các tập con gồm đúng k phần tử của tập

$$\{1, 2, \dots, n\}.$$

Mỗi tập con k phần tử có thể biểu diễn dưới dạng

$$\{x_1, x_2, \dots, x_k\}$$

với

$$1 \leq x_1 < x_2 < \cdots < x_k \leq n.$$

Số lượng các tập con như vậy là

$$\binom{n}{k}.$$

1. Phương pháp sinh tuần tự

Ý tưởng là xem mỗi tập con k phần tử như một tổ hợp chập k của n phần tử, được biểu diễn bởi dãy tăng

$$x_1 < x_2 < \dots < x_k.$$

Ta bắt đầu từ tổ hợp đầu tiên

$$1, 2, \dots, k$$

và sinh dần các tổ hợp kế tiếp cho đến tổ hợp cuối cùng

$$n - k + 1, n - k + 2, \dots, n.$$

Quy tắc sinh tổ hợp kế tiếp

Giả sử đang có tổ hợp

$$x_1, x_2, \dots, x_k.$$

Để sinh tổ hợp kế tiếp, ta làm như sau:

- Tìm từ phải sang trái vị trí i đầu tiên sao cho

$$x_i < n - k + i.$$

- Tăng x_i lên 1.
- Với mọi $j = i + 1, i + 2, \dots, k$, đặt

$$x_j = x_{j-1} + 1.$$

Nếu không tìm được vị trí i nào như trên thì tổ hợp hiện tại là tổ hợp cuối cùng, và ta dừng.

Ví dụ với $n = 5, k = 3$

Các tổ hợp được sinh lần lượt là:

$$123, 124, 125, 134, 135, 145, 234, 235, 245, 345.$$

Tương ứng với các tập con:

$$\{1, 2, 3\}, \{1, 2, 4\}, \{1, 2, 5\}, \{1, 3, 4\}, \{1, 3, 5\}, \{1, 4, 5\}, \{2, 3, 4\}, \{2, 3, 5\}, \{2, 4, 5\}, \{3, 4, 5\}.$$

Thuật toán

Khởi tạo $x_i = i$ với $i = 1, 2, \dots, k$

Lặp:

 In ra tổ hợp $x_1, x_2, \dots, x_k$

 Tìm i từ k về 1 sao cho $x_i < n - k + i$

 Nếu không tìm thấy thì dừng

 Đặt $x_i = x_i + 1$

 Với $j = i + 1, \dots, k$: đặt $x_j = x_{j-1} + 1$

2. Phương pháp quay lui

Ý tưởng của phương pháp quay lui là xây dựng dần tổ hợp tăng

$$x_1 < x_2 < \dots < x_k.$$

Ta chọn từng phần tử từ trái sang phải, bảo đảm thứ tự tăng để không lặp.

Gọi Try(i) là thủ tục chọn phần tử thứ i của tổ hợp.

- Nếu đang chọn x_i , thì giá trị của x_i phải lớn hơn x_{i-1} .
- Đồng thời, cần chứa đủ giá trị cho các vị trí phía sau, nên

$$x_i \leq n - k + i.$$

Vì thế, nếu đã có x_{i-1} , ta thử các giá trị:

$$x_i = x_{i-1} + 1, x_{i-1} + 2, \dots, n - k + i.$$

Khi đã chọn đủ k phần tử thì in ra một tập con.

Thuật toán

Try(i):

For v từ $x[i-1]+1$ đến $n-k+i$:

$x[i] = v$

Nếu $i = k$ thì in ra $x_1, x_2, \dots, x_k$

Ngược lại gọi Try($i+1$)

Khởi tạo $x_0 = 0$, sau đó gọi Try(1).

Ví dụ với $n = 5, k = 3$

Ta chọn:

$$x_1 \in \{1, 2, 3\}.$$

- Nếu $x_1 = 1$, thì $x_2 \in \{2, 3, 4\}$.
- Nếu $x_2 = 2$, thì $x_3 \in \{3, 4, 5\}$, thu được:

$$\{1, 2, 3\}, \{1, 2, 4\}, \{1, 2, 5\}.$$

- Nếu $x_2 = 3$, thì được:

$$\{1, 3, 4\}, \{1, 3, 5\}.$$

- Nếu $x_2 = 4$, thì được:

$$\{1, 4, 5\}.$$

Tiếp tục tương tự với $x_1 = 2$ và $x_1 = 3$, ta thu được toàn bộ các tập con 3 phần tử của $\{1, 2, 3, 4, 5\}$.

3. Nhận xét

Các tập con k phần tử của $\{1, 2, \dots, n\}$ chính là các tổ hợp chập k của n phần tử. Có thể liệt kê chúng bằng:

- phương pháp sinh tuần tự: sinh từ tổ hợp đầu tiên đến tổ hợp cuối cùng;
- phương pháp quay lui: chọn dần từng phần tử theo thứ tự tăng.

Cả hai phương pháp đều đúng và cho ra toàn bộ các tập con cần tìm.

Câu 3. Liệt kê các hoán vị của $\{1, 2, \dots, n\}$

Ta cần liệt kê tất cả các hoán vị của tập

$$\{1, 2, \dots, n\}.$$

Mỗi hoán vị là một cách sắp xếp n phần tử theo một thứ tự nào đó, có thể biểu diễn dưới dạng

$$x_1 x_2 \cdots x_n$$

trong đó $x_1, x_2, \dots, x_n$ là một hoán vị của các số $1, 2, \dots, n$.

Số lượng các hoán vị của n phần tử là

$$n!.$$

1. Phương pháp sinh tuần tự

Ý tưởng của phương pháp sinh tuần tự là bắt đầu từ hoán vị đầu tiên theo thứ tự từ điển:

$$1\ 2\ 3\ \cdots\ n$$

và sinh dần hoán vị kế tiếp cho đến hoán vị cuối cùng:

$$n\ (n-1)\ \cdots\ 2\ 1.$$

Quy tắc sinh hoán vị kế tiếp

Giả sử đang có hoán vị

$$x_1 x_2 \cdots x_n.$$

Để sinh hoán vị kế tiếp theo thứ tự từ điển, ta thực hiện như sau:

- Tìm từ phải sang trái chỉ số i lớn nhất sao cho

$$x_i < x_{i+1}.$$

- Nếu không tồn tại chỉ số như vậy thì hoán vị hiện tại là hoán vị cuối cùng, ta dừng.
- Tìm từ phải sang trái chỉ số j lớn nhất sao cho

$$x_j > x_i.$$

- Đổi chỗ x_i và x_j .
- Đảo ngược đoạn cuối

$$x_{i+1}, x_{i+2}, \dots, x_n.$$

Sau các bước trên, ta thu được hoán vị kế tiếp.

Ví dụ với $n = 3$

Bắt đầu từ hoán vị đầu tiên 123.

Hoán vị hiện tại	i	j	Sau khi đổi chỗ	Hoán vị kế tiếp
123	2	3	132	132
132	1	3	231	213
213	2	3	231	231
231	1	2	321	312
312	2	3	321	321
321	không có	—	—	dừng

Thuật toán

Khởi tạo $x_1 = 1, x_2 = 2, \dots, x_n = n$

Lặp:

```
In ra  $x_1x_2\dots x_n$   
Tìm  $i$  lớn nhất sao cho  $x_i < x_{i+1}$   
Nếu không có  $i$  thì dừng  
Tìm  $j$  lớn nhất sao cho  $x_j > x_i$   
Đổi chỗ  $x_i$  và  $x_j$   
Đảo ngược đoạn từ  $i+1$  đến  $n$ 
```

2. Phương pháp quay lui

Ý tưởng của phương pháp quay lui là xây dựng hoán vị từng vị trí một.

- Ở vị trí thứ 1, ta chọn một trong các số từ 1 đến n .
- Ở vị trí thứ 2, ta chọn một số chưa dùng.
- Tiếp tục như vậy cho đến vị trí thứ n .

Mỗi khi đã chọn đủ n vị trí thì ta thu được một hoán vị hoàn chỉnh và in ra.

Mô hình đệ quy

Gọi $\text{Try}(k)$ là thủ tục chọn phần tử cho vị trí thứ k .

- Với mỗi giá trị $v \in \{1, 2, \dots, n\}$ chưa được sử dụng, ta gán

$$x_k = v.$$

- Đánh dấu v đã được dùng.
- Nếu $k = n$ thì in ra hoán vị $x_1x_2\dots x_n$.
- Ngược lại gọi $\text{Try}(k+1)$.
- Sau đó bỏ đánh dấu để thử giá trị khác.

Thuật toán

Try(k) :

For v từ 1 đến n:

Nếu v chưa dùng thì

$x[k] = v$

Đánh dấu v đã dùng

Nếu $k = n$ thì in ra $x_1, x_2, \dots, x_n$

Ngược lại gọi Try(k+1)

Bỏ đánh dấu v

Ban đầu chưa có số nào được dùng và gọi Try(1).

Ví dụ với $n = 3$

Ta chọn:

$$x_1 \in \{1, 2, 3\}.$$

- Nếu $x_1 = 1$, thì x_2 có thể là 2 hoặc 3, tương ứng sinh ra:

123, 132.

- Nếu $x_1 = 2$, thì sinh ra:

213, 231.

- Nếu $x_1 = 3$, thì sinh ra:

312, 321.

Do đó thu được toàn bộ $3! = 6$ hoán vị.

3. Nhận xét

Có thể liệt kê các hoán vị của $\{1, 2, \dots, n\}$ bằng:

- phương pháp sinh tuần tự: sinh từ hoán vị đầu tiên đến hoán vị cuối cùng theo thứ tự từ điển;
- phương pháp quay lui: chọn lần lượt từng phần tử chưa sử dụng để đưa vào các vị trí của hoán vị.

Cả hai phương pháp đều cho ra đầy đủ toàn bộ $n!$ hoán vị.

Câu 4. Liệt kê các cách chia số tự nhiên n thành tổng các số nhỏ hơn n

Ta cần liệt kê các cách biểu diễn số tự nhiên n dưới dạng tổng các số tự nhiên dương nhỏ hơn n , tức là

$$n = a_1 + a_2 + \dots + a_m$$

trong đó

$$a_1 \geq a_2 \geq \dots \geq a_m \geq 1, \quad a_i < n.$$

Mỗi cách viết như vậy được gọi là một *phân hoạch* của số n .

Việc yêu cầu

$$a_1 \geq a_2 \geq \cdots \geq a_m$$

giúp tránh lặp. Chẳng hạn,

$$4 = 3 + 1$$

và

$$4 = 1 + 3$$

được xem là cùng một cách chia, nên ta chỉ giữ một dạng chuẩn là viết các số hạng theo thứ tự không tăng.

Ví dụ

Với $n = 4$, các cách chia là:

$$4 = 3 + 1,$$

$$4 = 2 + 2,$$

$$4 = 2 + 1 + 1,$$

$$4 = 1 + 1 + 1 + 1.$$

Với $n = 5$, các cách chia là:

$$5 = 4 + 1,$$

$$5 = 3 + 2,$$

$$5 = 3 + 1 + 1,$$

$$5 = 2 + 2 + 1,$$

$$5 = 2 + 1 + 1 + 1,$$

$$5 = 1 + 1 + 1 + 1 + 1.$$

1. Phương pháp sinh tuần tự

Ta biểu diễn một phân hoạch của n dưới dạng dãy không tăng

$$a_1, a_2, \dots, a_m.$$

Phân hoạch đầu tiên là

$$n = (n - 1) + 1,$$

và phân hoạch cuối cùng là

$$n = 1 + 1 + \cdots + 1.$$

Ý tưởng của thuật toán sinh tuần tự là từ một phân hoạch hiện tại, tạo ra phân hoạch kế tiếp bằng cách giảm một phần tử thích hợp rồi phân phối lại phần còn lại sao cho dãy vẫn không tăng.

Quy tắc sinh phân hoạch kế tiếp

Giả sử đang có phân hoạch

$$n = a_1 + a_2 + \cdots + a_m \quad \text{với } a_1 \geq a_2 \geq \cdots \geq a_m \geq 1.$$

Ta thực hiện:

- Tìm từ phải sang trái phần tử đầu tiên lớn hơn 1, giả sử là a_i .
- Giảm a_i đi 1:

$$a_i := a_i - 1.$$

- Gom tất cả phần còn lại ở phía sau thành một tổng R .
- Chia R thành các phần tử không lớn hơn a_i , càng nhiều phần tử bằng a_i càng tốt, phần dư (nếu có) đặt ở cuối.

Nếu mọi phần tử đều bằng 1 thì dừng.

Ví dụ với $n = 5$

Ta có dãy các phân hoạch:

$$\begin{aligned} &4 + 1, \\ &3 + 2, \\ &3 + 1 + 1, \\ &2 + 2 + 1, \\ &2 + 1 + 1 + 1, \\ &1 + 1 + 1 + 1 + 1. \end{aligned}$$

Giải thích một vài bước:

- Từ $4 + 1$: phần tử lớn hơn 1 cuối cùng là 4, giảm xuống còn 3, phần còn lại là 2, nên được

$$3 + 2.$$

- Từ $3 + 2$: phần tử lớn hơn 1 cuối cùng là 2, giảm xuống còn 1, phần còn lại là $1 + 1$, nên được

$$3 + 1 + 1.$$

- Từ $3 + 1 + 1$: phần tử lớn hơn 1 cuối cùng là 3, giảm xuống còn 2, phần còn lại là $2 + 1$, nên được

$$2 + 2 + 1.$$

Thuật toán

Khởi tạo phân hoạch đầu tiên: $n = (n-1) + 1$

Lặp:

In ra phân hoạch hiện tại

Tìm từ phải sang trái phần tử đầu tiên > 1

Nếu không có thì dừng

Giảm phần tử đó đi 1

Cộng các phần tử phía sau lại thành R

Chia R thành các phần tử không lớn hơn phần tử vừa giảm

2. Phương pháp quay lui

Phương pháp quay lui dễ hiểu hơn đối với bài toán này.

Ý tưởng là xây dựng dần phân hoạch:

$$n = a_1 + a_2 + \cdots + a_m$$

sao cho các số hạng được chọn theo thứ tự không tăng:

$$a_1 \geq a_2 \geq \cdots \geq a_m.$$

Tại mỗi bước:

- giả sử còn lại tổng cần phân tích là S ;
- ta chọn số hạng tiếp theo a_i trong đoạn từ 1 đến một cận trên M ;
- để giữ thứ tự không tăng, ta luôn yêu cầu

$$a_i \leq a_{i-1}.$$

Ban đầu, vì mỗi số hạng phải nhỏ hơn n , nên số hạng đầu tiên chỉ có thể chọn từ 1 đến $n - 1$.

Mô hình đệ quy

Gọi $\text{Try}(S, M)$ là thủ tục liệt kê các cách phân hoạch số S thành tổng các số không vượt quá M .

- Nếu $S = 0$ thì ta đã thu được một phân hoạch hợp lệ, in ra kết quả.
- Nếu $S > 0$, với mỗi giá trị

$$v = \min(S, M), \min(S, M) - 1, \dots, 1,$$

ta chọn tiếp một số hạng bằng v , rồi gọi đệ quy cho phần còn lại:

$$\text{Try}(S - v, v).$$

Việc truyền tiếp cận trên bằng v bảo đảm dãy thu được luôn không tăng.

Thuật toán

$\text{Try}(S, M)$:

```
Nếu  $S = 0$  thì  
    in ra phân hoạch hiện tại  
Ngược lại  
    For  $v$  từ  $\min(S, M)$  giảm dần đến 1:  
        chọn  $v$   
        gọi  $\text{Try}(S-v, v)$   
        bỏ chọn  $v$ 
```

Ban đầu gọi:

$\text{Try}(n, n-1)$

để bảo đảm số hạng đầu tiên nhỏ hơn n .

Ví dụ với $n = 5$

Ban đầu gọi $\text{Try}(5, 4)$.

- Chọn 4, còn lại 1, được:

$$5 = 4 + 1.$$

- Chọn 3, còn lại 2:

- chọn tiếp 2, được

$$5 = 3 + 2;$$

- chọn tiếp 1, rồi tiếp tục chọn 1, được

$$5 = 3 + 1 + 1.$$

- Chọn 2, còn lại 3:

- chọn tiếp 2, rồi 1, được

$$5 = 2 + 2 + 1;$$

- chọn tiếp 1, 1, 1, được

$$5 = 2 + 1 + 1 + 1.$$

- Chọn 1 năm lần, được

$$5 = 1 + 1 + 1 + 1 + 1.$$

3. Nhận xét

Có thể liệt kê các cách chia số tự nhiên n thành tổng các số nhỏ hơn n bằng:

- phương pháp sinh tuần tự: từ một phân hoạch hiện tại sinh ra phân hoạch kế tiếp;
- phương pháp quay lui: chọn dần từng số hạng sao cho dãy các số hạng luôn không tăng.

Cả hai phương pháp đều liệt kê đúng và đầy đủ tất cả các phân hoạch của n thỏa mãn yêu cầu.

Câu 5. Liệt kê tất cả các xâu nhị phân độ dài n có duy nhất một dãy k bit 0 liên tiếp và duy nhất một dãy m bit 1 liên tiếp

Ta cần liệt kê các xâu nhị phân

$$x_1 x_2 \cdots x_n, \quad x_i \in \{0, 1\}$$

thỏa mãn đồng thời hai điều kiện:

- có đúng một đoạn gồm đúng k bit 0 liên tiếp;
- có đúng một đoạn gồm đúng m bit 1 liên tiếp.

Ở đây, “một dãy k bit 0 liên tiếp” được hiểu là một đoạn gồm đúng k số 0 liên tiếp, không nằm trong một đoạn 0 dài hơn. Tương tự, “một dãy m bit 1 liên tiếp” là một đoạn gồm đúng m số 1 liên tiếp, không nằm trong một đoạn 1 dài hơn.

Ví dụ, nếu $k = 2$ thì trong xâu

001001

ta có hai dãy đúng 2 bit 0 liên tiếp, nên xâu này không thỏa mãn điều kiện “duy nhất một dãy 2 bit 0 liên tiếp”.

2. Phương pháp sinh tuần tự

Phương pháp sinh tuần tự vẫn có thể áp dụng cho bài toán này.

Ý tưởng là:

- sinh lần lượt tất cả các xâu nhị phân độ dài n theo thứ tự từ điển, bắt đầu từ

$00 \dots 0$

và kết thúc ở

$11 \dots 1;$

- với mỗi xâu thu được, kiểm tra xem:

- có đúng một đoạn gồm đúng k số 0 liên tiếp hay không;
- có đúng một đoạn gồm đúng m số 1 liên tiếp hay không.

- nếu thỏa mãn đồng thời cả hai điều kiện thì in ra xâu đó.

Cách sinh xâu kế tiếp

Giả sử đang có xâu nhị phân

$x_1 x_2 \dots x_n.$

Để sinh xâu kế tiếp, ta:

- tìm từ phải sang trái vị trí đầu tiên i sao cho $x_i = 0$;
- đổi x_i thành 1;
- đặt tất cả các bit phía sau nó về 0.

Nếu không còn vị trí nào bằng 0 thì xâu hiện tại là

$11 \dots 1,$

và thuật toán dừng.

Thuật toán

Khởi tạo $x_1 = x_2 = \dots = x_n = 0$

Lặp:

Nếu Check(x) đúng thì in ra x

Tìm i từ n về 1 sao cho $x_i = 0$

Nếu không tìm thấy thì dừng

Đặt $x_i = 1$

Với mọi $j = i+1, \dots, n$: đặt $x_j = 0$

Nhận xét

Phương pháp sinh tuần tự là đúng vì nó sinh ra đầy đủ toàn bộ 2^n xâu nhị phân độ dài n , không lặp và không thiếu. Tuy nhiên, cách này thường kém thuận lợi hơn quay lui vì phải sinh toàn bộ các xâu rồi mới kiểm tra điều kiện.

3. Phương pháp quay lui

Phương pháp quay lui xây dựng xâu từ trái sang phải.

Gọi $\text{Try}(i)$ là thủ tục gán giá trị cho vị trí thứ i .

- Tại mỗi vị trí i , ta thử hai khả năng:

$$x_i = 0 \quad \text{hoặc} \quad x_i = 1.$$

- Nếu $i = n$ thì đã thu được một xâu nhị phân độ dài n .
- Khi đó, ta gọi thủ tục kiểm tra. Nếu xâu thỏa mãn yêu cầu thì in ra.

Thuật toán quay lui

$\text{Try}(i)$:

```
For v in {0,1}:  
    x[i] = v  
    Nếu i = n thì  
        nếu Check(x) đúng thì in ra x  
    Ngược lại  
        gọi Try(i+1)
```

Ban đầu gọi $\text{Try}(1)$.

Nhận xét

So với sinh tuần tự, phương pháp quay lui phù hợp hơn với bài toán có ràng buộc. Trong trường hợp cần tối ưu, ta có thể kiểm tra dần trong khi xây dựng xâu để cắt bỏ sớm những nhánh chắc chắn không thỏa mãn điều kiện.

4. Thuật toán kiểm tra điều kiện

Giả sử đã có một xâu nhị phân

$$x_1x_2 \cdots x_n.$$

Ta duyệt từ trái sang phải để tìm các đoạn liên tiếp gồm các bit giống nhau.

Gọi:

- cnt0 là số đoạn gồm đúng k số 0 liên tiếp;
- cnt1 là số đoạn gồm đúng m số 1 liên tiếp.

Cách làm:

- Bắt đầu từ vị trí $i = 1$.

- Xác định bit hiện tại là x_i .
- Đếm độ dài đoạn liên tiếp bắt đầu từ i gồm các bit bằng x_i .
- Nếu bit đó là 0 và độ dài đoạn đúng bằng k thì tăng `cnt0`.
- Nếu bit đó là 1 và độ dài đoạn đúng bằng m thì tăng `cnt1`.
- Chuyển sang đoạn kế tiếp.

Sau khi duyệt hết xâu:

$$\text{Check}(x) = \text{true} \iff (\text{cnt0} = 1) \text{ và } (\text{cnt1} = 1).$$

Thuật toán kiểm tra

`Check(x)`:

```

cnt0 = 0
cnt1 = 0
i = 1
While i <= n:
    j = i
    While j <= n and x[j] = x[i]:
        j = j + 1
    len = j - i
    Nếu x[i] = 0 và len = k thì cnt0 = cnt0 + 1
    Nếu x[i] = 1 và len = m thì cnt1 = cnt1 + 1
    i = j
Nếu cnt0 = 1 và cnt1 = 1 thì return true
Ngược lại return false

```

5. Kết luận

Có thể liệt kê các xâu nhị phân độ dài n có duy nhất một dãy k bit 0 liên tiếp và duy nhất một dãy m bit 1 liên tiếp bằng:

- phương pháp sinh tuần tự: sinh toàn bộ các xâu nhị phân độ dài n , rồi kiểm tra từng xâu;
- phương pháp quay lui: xây dựng dần từng bit của xâu, sau đó kiểm tra điều kiện.

Cả hai phương pháp đều đúng. Tuy nhiên, với bài toán có điều kiện ràng buộc như bài này, phương pháp quay lui thường thuận lợi hơn và dễ mở rộng hơn.

Câu 6. Liệt kê các dãy con của dãy số A_n sao cho tổng các phần tử của dãy con đó đúng bằng k

Giả sử

$$A_n = (a_1, a_2, \dots, a_n).$$

Ta cần liệt kê tất cả các dãy con (hay tập con chỉ số) của dãy A_n sao cho tổng các phần tử được chọn đúng bằng k .

Mỗi dãy con có thể biểu diễn bằng một vectơ nhị phân

$$x_1, x_2, \dots, x_n, \quad x_i \in \{0, 1\},$$

trong đó:

- $x_i = 1$ nếu chọn phần tử a_i ;
- $x_i = 0$ nếu không chọn phần tử a_i .

Khi đó tổng các phần tử của dãy con tương ứng là

$$x_1 a_1 + x_2 a_2 + \dots + x_n a_n.$$

Ta cần liệt kê các vectơ nhị phân thỏa mãn

$$x_1 a_1 + x_2 a_2 + \dots + x_n a_n = k.$$

1. Phương pháp sinh tuần tự

Ta sinh lần lượt tất cả các vectơ nhị phân độ dài n :

$$00 \dots 0, 00 \dots 1, \dots, 11 \dots 1.$$

Mỗi vectơ nhị phân tương ứng với một cách chọn dãy con của A_n .

Với mỗi vectơ

$$x_1, x_2, \dots, x_n,$$

ta tính tổng

$$S = x_1 a_1 + x_2 a_2 + \dots + x_n a_n.$$

Nếu

$$S = k$$

thì in ra dãy con tương ứng.

Cách sinh vectơ nhị phân kế tiếp

Giả sử đang có xâu nhị phân

$$x_1 x_2 \dots x_n.$$

Để sinh xâu kế tiếp, ta:

- tìm từ phải sang trái vị trí đầu tiên i sao cho $x_i = 0$;
- đổi x_i thành 1;
- đặt tất cả các bit phía sau nó về 0.

Nếu không còn vị trí nào bằng 0 thì dừng.

Thuật toán

Khởi tạo $x_1 = x_2 = \dots = x_n = 0$

Lặp:

Tính $S = x_1 \cdot a_1 + x_2 \cdot a_2 + \dots + x_n \cdot a_n$
Nếu $S = k$ thì in ra dãy con tương ứng
Tìm i từ n về 1 sao cho $x_i = 0$
Nếu không tìm thấy thì dừng
Đặt $x_i = 1$
Với mọi $j = i+1, \dots, n$: đặt $x_j = 0$

Nhận xét

Phương pháp sinh tuần tự sinh ra đầy đủ tất cả 2^n cách chọn dãy con. Do đó, mọi dãy con có tổng bằng k đều sẽ được phát hiện.

2. Phương pháp quay lui

Phương pháp quay lui xây dựng dần quyết định chọn hay không chọn từng phần tử.

Gọi $\text{Try}(i)$ là thủ tục xét phần tử thứ i .

Tại mỗi bước, có hai khả năng:

- chọn a_i ;
- không chọn a_i .

Sau khi xét hết n phần tử, nếu tổng các phần tử đã chọn đúng bằng k thì in ra dãy con tương ứng.

Thuật toán quay lui cơ bản

$\text{Try}(i)$:

For v in $\{0,1\}$:
 $x[i] = v$
 Nếu $i = n$ thì
 tính $S = x_1 \cdot a_1 + x_2 \cdot a_2 + \dots + x_n \cdot a_n$
 nếu $S = k$ thì in ra dãy con tương ứng
 Ngược lại
 gọi $\text{Try}(i+1)$

Ban đầu gọi $\text{Try}(1)$.

Dạng quay lui có tích lũy tổng

Ta có thể viết tốt hơn bằng cách truyền theo tổng tạm thời.

Gọi $\text{Try}(i, S)$ là thủ tục xét từ vị trí i trở đi, trong đó S là tổng các phần tử đã chọn trước đó.

- Nếu $i > n$:
 - nếu $S = k$ thì in ra lời giải;

- ngược lại loại bỏ.
- Nếu $i \leq n$, ta thử:
 - không chọn a_i : gọi Try($i+1$, S);
 - chọn a_i : gọi Try($i+1$, S+ a_i).

Thuật toán

```

Try(i, S):
  Nếu i > n thì
    Nếu S = k thì in ra dãy con đã chọn
  Ngược lại:
    x[i] = 0
    gọi Try(i+1, S)

    x[i] = 1
    gọi Try(i+1, S + a[i])
  
```

Ban đầu gọi Try(1, 0).

3. Ví dụ

Xét dãy

$$A_4 = (1, 2, 3, 4), \quad k = 5.$$

Các dãy con có tổng đúng bằng 5 là:

$$(1, 4), \quad (2, 3).$$

Nếu biểu diễn bằng vectơ chọn, ta có:

$$(1, 0, 0, 1)$$

tương ứng với dãy con (1, 4), và

$$(0, 1, 1, 0)$$

tương ứng với dãy con (2, 3).

4. Kết luận

- Phương pháp sinh tuần tự đơn giản vì chỉ cần sinh toàn bộ các xâu nhị phân độ dài n rồi kiểm tra tổng.
- Phương pháp quay lui tự nhiên hơn vì mỗi bước chính là quyết định chọn hoặc không chọn một phần tử.
- Nếu các phần tử của dãy đều không âm, ta có thể tối ưu thêm trong quay lui: nếu tổng tạm thời đã lớn hơn k thì không cần xét tiếp nhánh đó.

Câu 7. Liệt kê tất cả các dãy con k phần tử của dãy số A_n sao cho tổng các phần tử của dãy con đó đúng bằng B

Giả sử

$$A_n = (a_1, a_2, \dots, a_n).$$

Ta cần liệt kê tất cả các dãy con gồm đúng k phần tử của dãy A_n sao cho tổng các phần tử được chọn bằng B .

Mỗi dãy con cần tìm có thể biểu diễn bằng bộ chỉ số

$$1 \leq i_1 < i_2 < \dots < i_k \leq n$$

sao cho

$$a_{i_1} + a_{i_2} + \dots + a_{i_k} = B.$$

1. Phương pháp sinh tuần tự

Ta xem mỗi dãy con k phần tử như một tổ hợp chập k của n vị trí:

$$i_1 < i_2 < \dots < i_k.$$

Vì vậy, bài toán trở thành: sinh tất cả các tổ hợp chập k của $\{1, 2, \dots, n\}$, rồi kiểm tra tổng các phần tử tương ứng.

Cách sinh tổ hợp kế tiếp

Khởi đầu từ tổ hợp đầu tiên:

$$1, 2, \dots, k.$$

Tổ hợp cuối cùng là:

$$n - k + 1, n - k + 2, \dots, n.$$

Giả sử đang có tổ hợp

$$i_1, i_2, \dots, i_k.$$

Để sinh tổ hợp kế tiếp, ta:

- tìm từ phải sang trái vị trí t đầu tiên sao cho

$$i_t < n - k + t;$$

- tăng i_t lên 1;
- với mọi vị trí phía sau, đặt

$$i_{t+1} = i_t + 1, \quad i_{t+2} = i_{t+1} + 1, \dots$$

Nếu không tìm được vị trí như vậy thì dừng.

Thuật toán

Khởi tạo $i[j] = j$ với $j = 1, 2, \dots, k$

Lặp:

Tính $S = a[i_1] + a[i_2] + \dots + a[i_k]$
Nếu $S = B$ thì in ra dãy con tương ứng

Tìm t từ k về 1 sao cho $i[t] < n-k+t$
Nếu không tìm thấy thì dừng

Đặt $i[t] = i[t] + 1$
Với $j = t+1, \dots, k$:
 $i[j] = i[j-1] + 1$

Nhận xét

Phương pháp sinh tuần tự sinh ra đầy đủ mọi cách chọn đúng k vị trí trong n vị trí. Do đó, mọi dãy con k phần tử có tổng bằng B đều sẽ được phát hiện.

2. Phương pháp quay lui

Phương pháp quay lui xây dựng dần dãy chỉ số

$$i_1 < i_2 < \dots < i_k.$$

Gọi $\text{Try}(t)$ là thủ tục chọn phần tử thứ t của dãy con.

- Khi đã chọn được

$$i_1, i_2, \dots, i_{t-1},$$

ta chọn tiếp i_t sao cho

$$i_t > i_{t-1}.$$

- Đồng thời phải chứa đủ vị trí cho các phần tử phía sau, nên

$$i_t \leq n - k + t.$$

Sau khi chọn đủ k chỉ số, ta tính tổng các phần tử tương ứng. Nếu tổng đúng bằng B thì in ra dãy con.

Thuật toán quay lui cơ bản

$\text{Try}(t)$:

For v từ $i[t-1] + 1$ đến $n-k+t$:
 $i[t] = v$
 Nếu $t = k$ thì
 tính $S = a[i_1] + a[i_2] + \dots + a[i_k]$
 nếu $S = B$ thì in ra dãy con
 Ngược lại
 gọi $\text{Try}(t+1)$

Khởi tạo $i_0 = 0$, sau đó gọi $\text{Try}(1)$.

Dạng quay lui có tích lũy tổng

Ta có thể cải tiến bằng cách truyền theo tổng tạm thời.

Gọi $\text{Try}(t, \text{last}, S)$ là thủ tục:

- đã chọn được $t - 1$ phần tử;
- chỉ số cuối cùng đã chọn là last ;
- tổng hiện tại là S .

Khi đó:

- nếu $t > k$ thì:
 - nếu $S = B$ thì in ra lời giải;
 - ngược lại loại bỏ;
- nếu $t \leq k$, ta thử chọn chỉ số tiếp theo từ

$$\text{last} + 1, \text{last} + 2, \dots, n - k + t.$$

Thuật toán

$\text{Try}(t, \text{last}, S)$:

Nếu $t > k$ thì

Nếu $S = B$ thì in ra dãy con đã chọn

Ngược lại:

For v từ $\text{last} + 1$ đến $n - k + t$:

$i[t] = v$

gọi $\text{Try}(t+1, v, S + a[v])$

Ban đầu gọi $\text{Try}(1, 0, 0)$.

3. Ví dụ

Xét dãy

$$A_5 = (1, 2, 3, 4, 5), \quad k = 2, \quad B = 5.$$

Ta cần chọn các dãy con gồm đúng 2 phần tử có tổng bằng 5.

Các khả năng là:

$$(1, 4), \quad (2, 3).$$

Tương ứng với chỉ số:

$$(1, 4), \quad (2, 3).$$

Vì

$$a_1 + a_4 = 1 + 4 = 5, \quad a_2 + a_3 = 2 + 3 = 5.$$

4. Nhận xét

Để liệt kê tất cả các dãy con k phần tử của dãy số A_n có tổng đúng bằng B , ta có thể:

- dùng phương pháp sinh tuần tự: sinh toàn bộ các tổ hợp chập k của n vị trí rồi kiểm tra tổng;
- dùng phương pháp quay lui: chọn dần các chỉ số tăng và kiểm tra tổng sau khi đủ k phần tử.

Cả hai phương pháp đều liệt kê đúng và đầy đủ các dãy con cần tìm.

Câu 8. Liệt kê tất cả các cách chọn trên mỗi hàng, mỗi cột khác nhau các phần tử của ma trận vuông A cấp n sao cho tổng các phần tử đó đúng bằng K

Giả sử

$$A = (a_{ij})_{n \times n}$$

là một ma trận vuông cấp n .

Ta cần liệt kê tất cả các cách chọn đúng n phần tử của ma trận sao cho:

- mỗi hàng chọn đúng một phần tử;
- mỗi cột chọn đúng một phần tử;
- tổng các phần tử được chọn đúng bằng K .

1. Mô hình bài toán

Vì mỗi hàng chọn đúng một phần tử và mỗi cột cũng chỉ được chọn đúng một lần, nên nếu ở hàng i ta chọn phần tử thuộc cột x_i , thì dãy

$$x_1, x_2, \dots, x_n$$

phải là một hoán vị của tập

$$\{1, 2, \dots, n\}.$$

Khi đó, tổng các phần tử được chọn là

$$a_{1x_1} + a_{2x_2} + \dots + a_{nx_n}.$$

Bài toán trở thành: liệt kê tất cả các hoán vị

$$x_1, x_2, \dots, x_n$$

của $\{1, 2, \dots, n\}$ sao cho

$$a_{1x_1} + a_{2x_2} + \dots + a_{nx_n} = K.$$

2. Phương pháp sinh tuần tự

Ta sinh lần lượt tất cả các hoán vị của

$$1, 2, \dots, n.$$

Mỗi hoán vị

$$x_1, x_2, \dots, x_n$$

tương ứng với một cách chọn phần tử:

$$a_{1x_1}, a_{2x_2}, \dots, a_{nx_n}.$$

Với mỗi hoán vị, ta tính tổng

$$S = a_{1x_1} + a_{2x_2} + \dots + a_{nx_n}.$$

Nếu

$$S = K$$

thì in ra cách chọn tương ứng.

Cách sinh hoán vị kế tiếp

Giả sử đang có hoán vị

$$x_1 x_2 \dots x_n.$$

Để sinh hoán vị kế tiếp theo thứ tự từ điển, ta thực hiện:

- tìm từ phải sang trái vị trí lớn nhất i sao cho

$$x_i < x_{i+1};$$

- nếu không tồn tại vị trí như vậy thì dừng;
- tìm từ phải sang trái vị trí lớn nhất j sao cho

$$x_j > x_i;$$

- đổi chỗ x_i và x_j ;
- đảo ngược đoạn

$$x_{i+1}, x_{i+2}, \dots, x_n.$$

Thuật toán

Khởi tạo $x[i] = i$ với $i = 1, 2, \dots, n$

Lặp:

Tính $S = a[1][x_1] + a[2][x_2] + \dots + a[n][x_n]$

Nếu $S = K$ thì in ra cách chọn tương ứng

Tìm i lớn nhất sao cho $x[i] < x[i+1]$

Nếu không có i thì dừng

Tìm j lớn nhất sao cho $x[j] > x[i]$

Đổi chỗ $x[i]$ và $x[j]$

Đảo ngược đoạn từ $i+1$ đến n

Nhận xét

Phương pháp sinh tuần tự sinh ra toàn bộ $n!$ hoán vị của $\{1, 2, \dots, n\}$. Do đó, mọi cách chọn thỏa mãn điều kiện mỗi hàng, mỗi cột khác nhau đều sẽ được xét đến.

3. Phương pháp quay lui

Phương pháp quay lui xây dựng dần hoán vị

$$x_1, x_2, \dots, x_n,$$

trong đó x_i là cột được chọn ở hàng i .

Ý tưởng:

- ở hàng 1, chọn một cột từ 1 đến n ;
- ở hàng 2, chọn một cột chưa dùng;
- tiếp tục như vậy cho đến hàng n .

Khi đã chọn đủ n hàng, nếu tổng các phần tử được chọn đúng bằng K thì in ra lời giải.

Thuật toán quay lui cơ bản

Gọi Try(i) là thủ tục chọn cột cho hàng thứ i .

Try(i):

For v từ 1 đến n :

Nếu cột v chưa dùng thì

$x[i] = v$

đánh dấu cột v đã dùng

Nếu $i = n$ thì

tính $S = a[1][x1] + a[2][x2] + \dots + a[n][xn]$

nếu $S = K$ thì in ra lời giải

Ngược lại

gọi Try($i+1$)

bỏ đánh dấu cột v

Ban đầu chưa có cột nào được dùng và gọi Try(1).

Dạng quay lui có tích lũy tổng

Ta có thể cải tiến bằng cách truyền theo tổng tạm thời.

Gọi Try(i, S) là thủ tục:

- đã chọn cột cho các hàng $1, 2, \dots, i-1$;
- tổng hiện tại là S .

Khi đó:

- nếu $i > n$ thì:

- nếu $S = K$ thì in ra lời giải;
- ngược lại loại bỏ;
- nếu $i \leq n$, ta thử chọn một cột v chưa dùng cho hàng i , rồi gọi

$\text{Try}(i + 1, S + a_{iv}).$

Thuật toán

$\text{Try}(i, S):$

```

    Nếu  $i > n$  thì
        Nếu  $S = K$  thì in ra cách chọn
    Ngược lại:
        For  $v$  từ 1 đến  $n$ :
            Nếu cột  $v$  chưa dùng thì
                 $x[i] = v$ 
                đánh dấu cột  $v$  đã dùng
                gọi  $\text{Try}(i+1, S + a[i][v])$ 
                bỏ đánh dấu cột  $v$ 
```

Ban đầu gọi $\text{Try}(1, 0)$.

Nhận xét

Phương pháp quay lui phù hợp hơn vì:

- mỗi bước chính là chọn một cột cho một hàng;
- dễ kiểm soát điều kiện “mỗi cột chỉ dùng một lần”;
- thuận tiện để tối ưu thêm nếu cần.

Nếu các phần tử của ma trận đều không âm, thì khi tổng tạm thời đã lớn hơn K , ta có thể dừng sớm nhánh đó.

4. Ví dụ

Xét ma trận

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 1 & 4 \\ 3 & 4 & 1 \end{pmatrix}, \quad K = 3.$$

Ta cần chọn mỗi hàng một phần tử, mỗi cột khác nhau, sao cho tổng bằng 3. Các hoán vị của $\{1, 2, 3\}$ là:

$(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1).$

Tính tổng tương ứng:

- $(1, 2, 3)$: chọn

$$a_{11}, a_{22}, a_{33}$$

có tổng

$$1 + 1 + 1 = 3;$$

- $(1, 3, 2)$: tổng là

$$1 + 4 + 4 = 9;$$

- $(2, 1, 3)$: tổng là

$$2 + 2 + 1 = 5;$$

- các trường hợp còn lại cũng không cho tổng bằng 3.

Vậy chỉ có một cách chọn thỏa mãn là:

$$a_{11}, a_{22}, a_{33}.$$

5. Kết luận

Để liệt kê tất cả các cách chọn trên mỗi hàng, mỗi cột khác nhau các phần tử của ma trận vuông A cấp n sao cho tổng đúng bằng K , ta có thể:

- dùng phương pháp sinh tuần tự: sinh toàn bộ các hoán vị của $\{1, 2, \dots, n\}$ rồi kiểm tra tổng;
- dùng phương pháp quay lui: chọn dần cột cho từng hàng, bảo đảm mỗi cột chỉ được chọn một lần.

Cả hai phương pháp đều đúng và liệt kê đầy đủ các lời giải của bài toán.